

# Modelos Generativos Profundos

## Clase 21: Modelos de difusión (parte III)

---

Fernando Fêtis Riquelme

Otoño, 2026

Facultad de Ciencias Físicas y Matemáticas  
Universidad de Chile

Generación condicional y guidance

Arquitecturas neuronales para DMs

Algunos DMs famosos

# Generación condicional y guidance

---

## Idea general

Al igual que toda red bayesiana, un modelo de difusión puede ser extendido para aprender **distribuciones condicionales**  $p_{\text{data}}(x|y)$ , donde  $y \sim p(y)$  es alguna variable condicionante. Para esto, es suficiente considerar un proceso backward condicional,

$$p_{\theta}(x_{0:T}|y) = p(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1}|x_t, y),$$

donde ahora las transiciones  $p_{\theta}(x_{t-1}|x_t, y) \sim \mathcal{N}(\mu_{\theta}(x_t, t, y), \sigma_q^2(t) \mathbf{I}_D)$  dependen del factor condicionante  $y$ .

En la práctica, esto se logra considerando a  $y$  como una entrada adicional a la red neuronal  $\mu_{\theta}$ , la cual puede ser inyectada de diversas formas, dependiendo de la naturaleza de  $y$ .



## Idea general

En su forma más simple,  $y \in \mathbb{R}^C$  es un vector, por ejemplo:

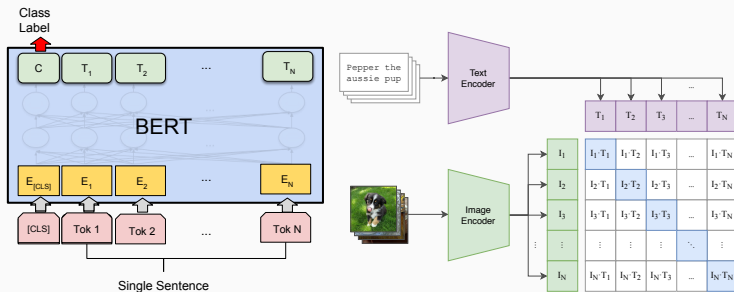
- Un vector one-hot que indica una clase.
- Un embedding de texto obtenido a partir de un modelo preentrenado (p.g. CLIP o BERT).

Si bien una estrategia simple es concatenar  $y$  a la entrada de la red neuronal, en la práctica se utilizan técnicas más sofisticadas.

Un método usual consiste en sumar el embedding temporal (asociado al tiempo de denoising  $t \in \{1, \dots, T\}$ ) y el embedding de la condición  $y \in \mathbb{R}^C$ , para luego inyectar esta suma en la red neuronal mediante **modulaciones tipo FiLM**.

# Modelos de embedding para texto

Para obtener una representación vectorial de un prompt, se pueden utilizar modelos preentrenados como **BERT** o **CLIP**.



Sin embargo, solo inyectar un embedding del prompt suele ser insuficiente para obtener una buena generación condicional. La técnica de **guidance** permite darle mayor peso a la condición durante el proceso de generación.

## Reparametrización score-prediction

Un modelo de difusión entrena una red neuronal  $\mu_\theta(x_t, t)$  para predecir la media  $\mu_q(x_0, x_t, t)$  de la distribución  $q(x_{t-1}|x_t, x_0)$ . La clase pasada se vieron formas equivalentes para esta cantidad:

- $\mu_q(x_0, x_t, t) = \left( \frac{c_1(t)}{\sqrt{\bar{\alpha}_t}} + c_2(t) \right) x_t - \frac{c_1(t)\sqrt{1-\bar{\alpha}_t}}{\sqrt{\bar{\alpha}_t}} \epsilon(x_0, x_t, t)$
- $\mu_q(x_0, x_t, t) = \left( \frac{c_1(t)}{\sqrt{\bar{\alpha}_t}} + c_2(t) \right) x_t + \frac{c_1(t)(1-\bar{\alpha}_t)}{\sqrt{\bar{\alpha}_t}} \nabla_{x_t} \log q(x_t)$

En particular, se tiene que

$$\epsilon(x_0, x_t, t) = -\sqrt{1 - \bar{\alpha}_t} \nabla_{x_t} \log q(x_t),$$

por lo que aprender el ruido o el score es, en esencia, el mismo problema.

Sin embargo, la formulación de score permite descomponer la señal de la condición sobre el proceso de generación.

# Classifier guidance

Dada la equivalencia anterior, un DM condicional necesita aprender el score condicional  $\nabla_{x_t} \log q(x_t|y)$ . Usando la regla de Bayes:

$$\underbrace{\nabla_{x_t} \log q(x_t|y)}_{\text{DM condicional}} = \nabla_{x_t} \log q(y|x_t) + \underbrace{\nabla_{x_t} \log q(x_t)}_{\text{DM incondicional}}$$

En consecuencia, si se conoce  $\nabla_{x_t} \log q(y|x_t)$ , se puede realizar generación condicional utilizando un DM incondicional.

Esta cantidad puede ser obtenida usando diferenciación automática sobre un **clasificador** entrenado para predecir  $q(y|x_t)$ .

# Classifier guidance

Más aún, es posible amplificar el efecto del condicionamiento utilizando un factor de escala  $s \geq 1$ :

$$\nabla_{x_t} \log \tilde{q}(x_t|y) = s \cdot \nabla_{x_t} \log q(y|x_t) + \nabla_{x_t} \log q(x_t).$$

Esta ponderación puede interpretarse como una disminución de **temperatura**, similar a lo que se hace en un LLM:

$$\tilde{q}(x_t|y) \propto q(y|x_t)^s q(x_t).$$

Esta técnica se denomina **classifier guidance**, y permite darle mayor peso a la condición y (usualmente un prompt o etiqueta de clase) durante el proceso de generación.

# Classifier-free guidance

Una limitación de la técnica anterior es que requiere entrenar un clasificador adicional  $q(y|x_t)$ , lo cual puede ser costoso.

Una alternativa más eficiente se obtiene al notar que

$$\nabla_{x_t} \log q(y|x_t) = \nabla_{x_t} \log q(x_t|y) - \nabla_{x_t} \log q(x_t),$$

por lo que se puede sustituir esta cantidad en CG para obtener

$$\nabla_{x_t} \log \tilde{q}(x_t|y) = s \cdot \underbrace{\nabla_{x_t} \log q(x_t|y)}_{\text{DM condicional}} + (1-s) \cdot \underbrace{\nabla_{x_t} \log q(x_t)}_{\text{DM incondicional}}.$$

Notar que se puede utilizar un mismo DM condicional para ambas cantidades ( $\nabla_{x_t} \log q(x_t)$  se obtiene al considerar  $y = \emptyset$ ).

## Classifier-free guidance

La técnica anterior se denomina **classifier-free guidance** y, a diferencia de CG, necesita entrenar un DM condicional:

$$\nabla_{x_t} \log \tilde{q}(x_t|y) = s \cdot \nabla_{x_t} \log q(x_t|y) + (1 - s) \cdot \nabla_{x_t} \log q(x_t).$$

Sin embargo, ya no es necesario entrenar un clasificador adicional, y se sigue teniendo la posibilidad de utilizar un guidance scale,  $s \geq 1$ , para darle más importancia a factor condicionante.

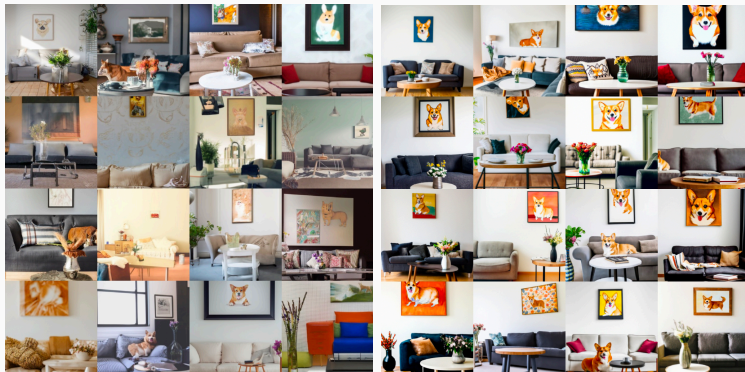
Por otro lado, la equivalencia  $\epsilon$ -score permite heredar la técnica de CFG directamente a DMs basados en  $\epsilon$ -prediction:

$$\tilde{\epsilon}(x_t, t, y) = s \cdot \epsilon(x_t, t, y) + (1 - s) \cdot \epsilon(x_t, t, \emptyset).$$

Esta es la forma más común de implementar CFG en la práctica.

# Classifier-free guidance

Generación condicional simple (izquierda) y con guidance (derecha).



**Prompt:** A cozy living room with a painting of a corgi on the wall above a couch and a round coffee table in front of a couch and a vase of flowers on a coffee table.

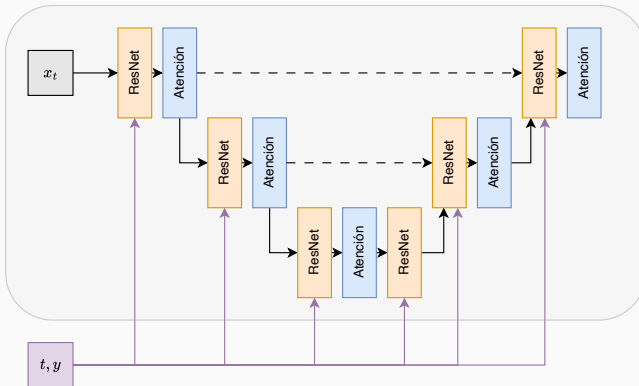


# Arquitecturas neuronales para DMs

---

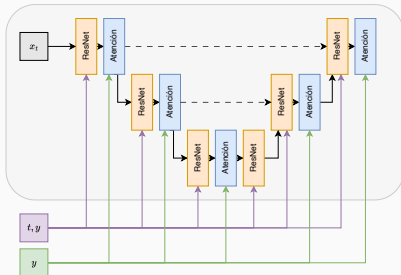
# U-Net

La arquitectura clásica en DMs es la arquitectura **U-Net con atención**. Si  $\text{emb}(t) + \text{emb}(y)$  es un vector, este se inyecta en la U-Net mediante modulaciones tipo FiLM en cada bloque convolucional.



# U-Net

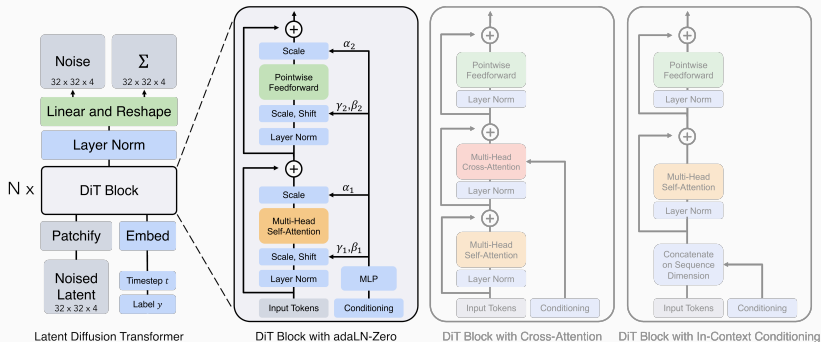
Si  $y$  es una secuencia de texto, también se puede considerar la secuencia de embeddings  $\{e_1, \dots, e_L\}$  obtenida a partir de un modelo preentrenado (p.g. T5). En este caso, se puede inyectar la información de  $y$  en la U-Net mediante **atención cruzada**.



Notar que se usan dos representaciones de  $y$ : un **embedding global** sumado al embedding temporal e inyectado por FiLM; y la **secuencia**  $\{e_1, \dots, e_L\}$ , inyectada por atención cruzada.

# Diffusion Transformer

Hoy en día, la arquitectura predominante en DMs (y flow matching) es la arquitectura **Diffusion Transformer (DiT)**, la cual puede verse como la arquitectura Vision Transformer (ViT) con capas de normalización moduladas por el tiempo y la condición.



## Algunos DMs famosos

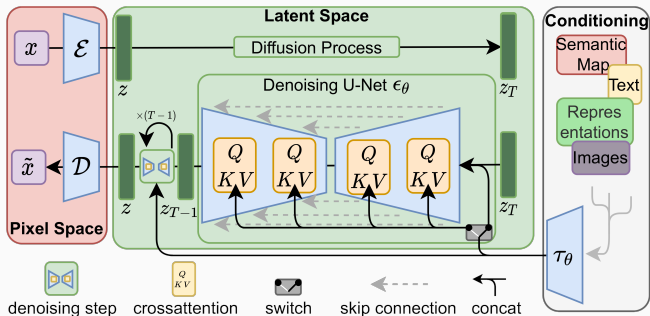
---



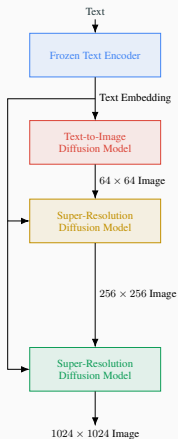
- DDPM fue el primer DM moderno.
- Sin embargo, los DMs fueron propuestos originalmente el 2015.
- Es un modelo incondicional, entrenado mediante  $\epsilon$ -prediction.
- Utiliza una U-Net simple como arquitectura neuronal.
- Su formulación se suele considerar como la formulación (discreta) estándar.

# Stable Diffusion

- Entrena un DM condicional en el espacio latente de un VAE.
- Utiliza una U-Net e inyecta el prompt mediante atención cruzada.
- Utiliza classifier-free guidance.



# Imagen



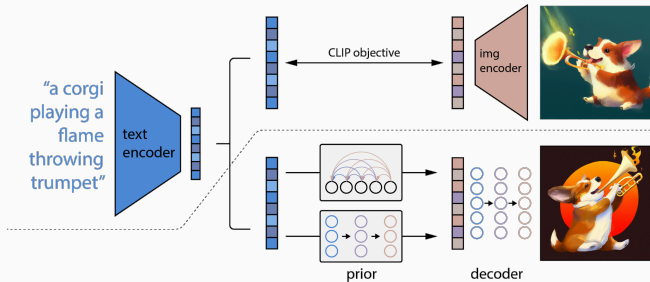
"A Golden Retriever dog wearing a blue checkered beret and red dotted turtleneck."



- Utiliza un esquema de cascada de DMs.
- Muestra que utilizar un LLM potente como text encoder es más importante que la arquitectura del DM.
- Se entrena con el enfoque  $x_0$ -prediction.



- Se entrena un DM (prior model) para generar un image CLIP embedding a partir del text CLIP embedding del prompt.
- Luego, se entrena otro DM (decoder model) para generar imágenes a partir de los image CLIP embeddings.
- Se puede mejorar considerablemente la calidad utilizando mejores captions durante el entrenamiento.



# Inpainting

La tarea de **inpainting** es un caso particular de generación condicional donde se busca completar una imagen a partir de una **máscara** que indica las regiones faltantes.



Para esto, se puede realizar un sampling usual, pero manteniendo fijas las regiones no enmascaradas.

En la próxima clase:

- Adaptadores para DMs.

# Modelos Generativos Profundos

## Clase 21: Modelos de difusión (parte III)